A Separation-Only Coding Scheme on $\omega_1^{\omega_1}$ with Ordinary Cohen Traces

Stefan Hoffelner

Working draft, May 15, 2026

Abstract

This note records a simplified version of the coding mechanism intended for the separation-only construction on $\omega_1^{\omega_1}$. The present version does not try to provide a reusable coding apparatus for reduction or uniformization. The basic coding block takes a set $x \subseteq \omega_1$, a bookkeeping-supplied container I_η, and an ordinary Cohen real $c \in 2^\omega$. The Cohen real produces an almost-disjoint trace $H_c \subseteq \omega$. Inside the container $I_\eta \cong \omega_1 \times \omega \times 2$, the characteristic function of x is written along the trace H_c by a branch/no-branch pattern through the corresponding blocks of the preliminary Suslin-tree apparatus. Each container is allowed to be used only finitely often, and different uses of the same container use different Cohen traces. This lets the construction be presented as a finite-support iteration of c.c.c. forcing notions. The recursive allowability scheme for separation is then obtained by the usual "force intersection if possible; otherwise code the appropriate side" rule.

Contents

1 Background and change of strategy **2**

2 Containers and ordinary Cohen traces **2**

3 The separation coding block **3**
3.1 The branch pattern associated with a trace 3
3.2 Coding the pattern into a real 3

4 Separation-only 0-allowability **4**

5 Recursive allowability for separation **5**

6 The separation iteration **6**
6.1 Bookkeeping 7
6.2 The dynamically defined allowability classes 7
6.3 The stage construction 8
6.4 No bad placements after activation 9
6.5 The separator associated with an active pair 10

7 Remaining proof obligations **11**

1 Background and change of strategy

The earlier approach tried to code a set $x \subseteq \omega_1$ by a real r_x and then to code r_x by the classical Baire-space coding predicate. This approach is too strong for the separation-only argument and creates unnecessary support problems: a real coding an arbitrary subset of ω_1 may carry information about ω_1 many earlier branch coordinates.

For the separation theorem we use a simpler one-shot mechanism. We code the characteristic function of x directly into a container of the Suslin-tree apparatus. Since each container is used only finitely often, ordinary Cohen reals suffice to generate almost-disjoint traces inside the container. This removes the need for the old hybrid product/iteration presentation using L-computed ω_1-Cohen coordinates. The construction becomes an ordinary finite-support iteration of c.c.c. forcings.

2 Containers and ordinary Cohen traces

Work over a preliminary model W, obtained from L by the fixed preliminary c.c.c. construction which supplies the branch apparatus associated with the canonical sequence $\vec{S}$ of Suslin trees. We do not force with the trees in this note. Rather, the branch information already available in W is used as raw material and is selectively coded into reals.

Fix in L a partition of the relevant Suslin-tree index set into containers

$$\langle I_\eta \mid \eta < \omega_2 \rangle, \qquad |I_\eta| = \omega_1.$$

For each $\eta < \omega_2$, fix an L-definable bijection

$$e_\eta : \omega_1 \times \omega \times 2 \longrightarrow I_\eta.$$

We write

$$S_{\eta,\xi,n,i} = S_{e_\eta(\xi,n,i)}$$

and let $b_{\eta,\xi,n,i}$ denote the corresponding branch object in the preliminary model W, when it is needed for coding.

Definition 2.1 (Ordinary Cohen trace)**.** *Let $c \in 2^\omega$. Fix a recursive injection*

$$\ulcorner . \urcorner : 2^{<\omega} \longrightarrow \omega.$$

The trace *of c is*

$$H_c = \{ \ulcorner c \restriction n \urcorner \mid n < \omega \} \subseteq \omega.$$

Lemma 2.2 (Trace almost-disjointness)**.** *If $c, d \in 2^\omega$ are distinct, then $H_c \cap H_d$ is finite.*

Proof. Let $N < \omega$ be least such that $c(N) \neq d(N)$. Then $c \restriction n \neq d \restriction m$ whenever $n, m > N$ and at least one of n, m is large enough to extend beyond the first disagreement in the appropriate way. Hence only finitely many finite binary sequences occur as initial segments of both c and d, and therefore $H_c \cap H_d$ is finite. □

Definition 2.3 (Finite-use record)**.** *A* finite-use record *is a function*

$$u : \omega_2 \longrightarrow [2^\omega]^{<\omega}$$

with finite values. It records which Cohen traces have already been used inside each container. A new use of I_η must use a Cohen real outside $u(\eta)$.

3 The separation coding block

The coding block is designed to code a tuple

$$t = (x, m, k, i),$$

where $x \subseteq \omega_1$, m, k code two Σ^1_1 predicates, and $i \in \{0, 1\}$ records the side. The container index η is supplied by the bookkeeping.

3.1 The branch pattern associated with a trace

Let $x \subseteq \omega_1$, $c \in 2^\omega$, and $\eta < \omega_2$. The intended branch pattern in I_η is the following:

$$\xi \in x \iff \text{for all but finitely many } n \in H_c, \text{ the branch } b_{\eta,\xi,n,1} \text{ is present in the code,}$$

and

$$\xi \notin x \iff \text{for all but finitely many } n \in H_c, \text{ the branch } b_{\eta,\xi,n,0} \text{ is present in the code.}$$

Equivalently, for each $\xi < \omega_1$ and for cofinitely many $n \in H_c$, the chosen side at the pair

$$\{S_{\eta,\xi,n,0}, S_{\eta,\xi,n,1}\}$$

records the bit $\chi_x(\xi)$.

The finite-exception formulation is essential. If a container is used finitely many times, the traces of the other uses have only finite intersection with H_c. Hence they can disturb only finitely many positions in each ξ-column.

3.2 Coding the pattern into a real

Fix in L an almost-disjoint family of reals

$$\mathcal{D} = \langle d_\alpha \mid \alpha < \omega_1 \rangle$$

used to code subsets of ω_1 by reals. Given x, c, η and the side tag (m, k, i), define a canonical subset

$$Y(\eta, c, x, m, k, i) \subseteq \omega_1$$

which codes:

(i) the header (η, m, k, i);

(ii) the Cohen real c, or equivalently the trace H_c;

(iii) for each $\xi < \omega_1$ and $n \in H_c$, the branch object $b_{\eta,\xi,n,\chi_x(\xi)}$;

(iv) the assertion that no other branch information in the same container is part of this code, except for finitely many harmless overlaps with other traces.

The exact coding of branch objects into a subset of ω_1 is fixed once and for all by the preliminary coding apparatus in W.

Definition 3.1 (The coding block $\mathbb{C}_\eta(\dot{x}, m, k, i)$)**.** *Let $\dot{x}$ be a name for a subset of ω_1, let m, k be codes for Σ^1_1 predicates, let $i < 2$, and let $\eta < \omega_2$. The coding block*

$$\mathbb{C}_\eta(\dot{x}, m, k, i)$$

is the two-step c.c.c. forcing

$$\mathrm{Add}(\omega, 1) * \dot{\mathbb{A}}_{\mathcal{D}}\big(\dot{Y}(\eta, \dot{c}, \dot{x}, m, k, i)\big),$$

where $\dot{c}$ is the Cohen real added by the first factor and $\dot{\mathbb{A}}_{\mathcal{D}}(\dot{Y})$ is the usual almost-disjoint coding forcing which adds a real R coding the subset $\dot{Y} \subseteq \omega_1$ with respect to $\mathcal{D}$.

Lemma 3.2 (The coding block is c.c.c.)**.** *Each forcing $\mathbb{C}_\eta(\dot{x}, m, k, i)$ is c.c.c.*

Proof. The first factor is ordinary Cohen forcing on ω. Over the Cohen extension, the second factor is the usual almost-disjoint coding forcing by the fixed family $\mathcal{D}$ of reals. This forcing is c.c.c.; conditions with the same finite stem are compatible after amalgamating their finite side conditions. Hence the two-step iteration is c.c.c. □

4 Separation-only 0-allowability

Definition 4.1 (0-allowable forcing, separation version)**.** *A forcing $\mathbb{P}$ is* 0-allowable for separation *if it has a finite-support iteration presentation*

$$\langle \mathbb{P}_\alpha, \dot{\mathbb{Q}}_\alpha \mid \alpha < \theta \rangle$$

for some $\theta \leq \omega_2$, such that each nontrivial iterand is of the form

$$\dot{\mathbb{Q}}_\alpha = \mathbb{C}_{\eta_\alpha}(\dot{x}_\alpha, m_\alpha, k_\alpha, i_\alpha),$$

and the following finite-use condition holds:

$$\big|\{\alpha < \theta \mid \eta_\alpha = \eta\}\big| < \omega \quad \textit{for every } \eta < \omega_2.$$

We denote the class of these forcings by Γ_0.

Remark 4.2. The finite-use condition is the replacement for the old freshness discipline. We no longer require a globally new container at every occurrence, but every container may be used only finitely many times. The different uses produce distinct ordinary Cohen reals and hence almost-disjoint traces inside the same container.

Lemma 4.3 (Finite products)**.** *The class Γ_0 is closed under finite products, up to the canonical finite-support presentation.*

Proof. Let $\mathbb{P}^0, \dots, \mathbb{P}^{r-1}$ be 0-allowable. Present their product as the finite-support iteration which interleaves the finitely many presentations. For a fixed container I_η, each factor uses I_η only finitely many times. Since there are only finitely many factors, the product presentation also uses I_η only finitely many times. Each iterand remains a c.c.c. coding block, and the product presentation is a finite-support iteration of such blocks. Hence the product is again in Γ_0. □

Lemma 4.4 (Finite-use no-unwanted-code lemma)**.** *Let $\mathbb{P} \in \Gamma_0$, let $G \subseteq \mathbb{P}$ be generic, and fix a container I_η. Suppose that I_η is used in the presentation of $\mathbb{P}$ at stages*

$$\alpha_0, \dots, \alpha_{r-1}$$

with Cohen reals

$$c_0, \dots, c_{r-1}.$$

If a real $R \in W[G]$ decodes a well-formed separation code with header (η, c, m, k, i), then either $c = c_j$ for some $j < r$, in which case the decoded pattern is the one intentionally written at stage α_j, or the alleged code fails on a tail of the trace H_c.

Proof sketch. Let c be distinct from all c_j. By Lemma 2.2,

$$H_c \cap \bigcup_{j<r} H_{c_j}$$

is finite. Therefore, outside finitely many positions of H_c, no branch data from an intentional use of I_η is available at the coordinates required by the alleged header (η, c, m, k, i). A well-formed code requires an eventual branch pattern along H_c in every ξ-column. Hence the computation fails on a tail of H_c.

If $c = c_j$ for some $j < r$, then the header points to the same coherent trace as stage α_j. The canonical definition of $Y(\eta, c_j, x_j, m_j, k_j, i_j)$ is injective in the tuple it codes. Thus the pattern read along H_{c_j} is exactly the pattern intentionally written at that stage. Finite overlaps with the other traces affect only finitely many positions and are ignored by the eventual decoding rule. □

5 Recursive allowability for separation

Let

$$\Gamma_0 \supseteq \Gamma_1 \supseteq \cdots \supseteq \Gamma_\alpha \supseteq \cdots$$

be the decreasing sequence of allowable classes used for the separation construction. The base class Γ_0 is Definition 4.1. The successor step records the separation rule relative to the previous class.

We fix a universal enumeration of Σ^1_1 predicates on 2^{ω_1}. For indices m, k and a parameter p, write

$$M_m(x, p), \qquad M_k(x, p)$$

for the corresponding Σ^1_1 assertions.

Definition 5.1 (Successor allowability rule)**.** *Assume Γ_α has been defined. A forcing is $\Gamma_{\alpha+1}$-allowable if it is obtained from a Γ_α-allowable forcing by applying the following rule at each bookkeeping request (x, m, k, p).*

(i) *If the current forcing already makes*

$$\exists z \big(M_m(z, p) \wedge M_k(z, p) \big),$$

then do nothing.

(ii) *Otherwise ask whether there is a Γ_α-allowable forcing which forces*

$$\exists z \big(M_m(z, p) \wedge M_k(z, p) \big).$$

If yes, perform such a forcing. The pair (m, k, p) is then neutralized: by upward absoluteness of Σ_1 statements over the relevant local structure, it remains intersecting in all later extensions.

(iii) *If no Γ_α-allowable forcing can force the two predicates to intersect, ask whether there is a Γ_α-allowable forcing which forces*

$$M_m(x, p).$$

If yes, do not perform that forcing. Instead choose a bookkeeping-supplied container I_η and append the coding block

$$\mathbb{C}_\eta(\check{x}, m, k, 0),$$

thereby coding the tuple $(x, m, k, p, 0)$ on the m-side.

(iv) *If no such forcing exists for the m-side, ask whether there is a Γ_α-allowable forcing which forces*

$$M_k(x, p).$$

If yes, do not perform that forcing. Instead append

$$\mathbb{C}_\eta(\check{x}, m, k, 1),$$

thereby coding the tuple $(x, m, k, p, 1)$ on the k-side.

(v) *If neither side can be forced, do nothing for this request.*

The same rule is added as a permanent obligation for future appearances of the same pair (m, k, p) with different points x'.

At limit stages λ, put

$$\Gamma_\lambda = \bigcap_{\alpha<\lambda} \Gamma_\alpha.$$

The final class is

$$\Gamma_\infty = \bigcap_{\alpha} \Gamma_\alpha.$$

Remark 5.2. Unlike the reduction and uniformization constructions, this recursive scheme is not intended to provide a general reusable coding mechanism. It is tailored to separation. Its only product requirement is the finite product closure needed in the contradiction argument below.

6 The separation iteration

In this section we describe the actual forcing construction which produces the separation property. The important point is that the classes of allowable forcings are not fixed in advance. They are defined along the run of the iteration. At stage α, the construction has already produced an allowability class Γ_α, determined by the separation requirements activated before α. The bookkeeping then presents a new request, and we either neutralize the relevant pair of Σ_1-sets by forcing them to intersect, or we add a new separation rule for this pair and code the current point on the side on which it can be forced to belong.

Throughout this section, Σ_1 means Σ_1 over the relevant $H(\omega_2)$-structure with the fixed coding apparatus as predicate. Equivalently, these are the generalized Σ^1_1-sets under the conventions established in the preliminary section. We use the notation

$$M_m(x, p), \qquad M_k(x, p)$$

for two such Σ_1-statements, with parameter $p \in H(\omega_2)$. The only absoluteness property used below is the upward absoluteness of Σ_1-truth between the intermediate forcing extensions occurring in the construction.

6.1 Bookkeeping

Fix in the ground model a bookkeeping function

$$F : \omega_2 \longrightarrow H(\omega_2)$$

with the following property. Every finite tuple of objects in $H(\omega_2)$, or more precisely every finite tuple of names for such objects appearing in some intermediate stage of the iteration, is listed by F unboundedly often. We use this in the standard way: at stage α, if $F(\alpha)$ is a code for a $\mathbb{P}_\alpha$-name for a tuple

$$(\dot{x}, \dot{p}, m, k),$$

then in the $\mathbb{P}_\alpha$-extension we write

$$x = \dot{x}^{G_\alpha}, \qquad p = \dot{p}^{G_\alpha},$$

and we regard (x, p, m, k) as the request handled at stage α. If $F(\alpha)$ does not decode such a request, the stage is trivial.

The indices m, k code Σ_1-definitions. The pair (m, k, p) will be called *neutralized* once the construction has forced

$$\exists z \, \big(M_m(z, p) \wedge M_k(z, p)\big).$$

After neutralization no further action for this pair is needed, because this intersection is preserved by upward absoluteness.

6.2 The dynamically defined allowability classes

We recursively define classes

$$\Gamma_0 \supseteq \Gamma_1 \supseteq \cdots \supseteq \Gamma_\alpha \supseteq \cdots$$

of separation-allowable forcings. Equivalently, the list of requirements grows with α, while the associated class of forcings becomes more restrictive. The base class Γ_0 is the class of finite-support c.c.c. iterations of the coding blocks from Section **??**, with the finite-use condition on containers.

At successor stages the following rule is added.

Definition 6.1 (One separation requirement)**.** *Suppose Γ_α has been defined, and suppose that at stage α the pair (m, k, p) is not yet neutralized and no Γ_α-allowable forcing can force*

$$\exists z \, \big(M_m(z, p) \wedge M_k(z, p)\big).$$

The requirement $\mathcal{R}(m, k, p, \alpha)$ says that whenever the bookkeeping later presents a point x' together with the same pair (m, k, p), we ask:

(i) *is there a Γ_α-allowable forcing which forces $M_m(x', p)$? If yes, then do not perform that forcing; instead code $(x', m, k, p, 0)$;*

(ii) *if not, is there a Γ_α-allowable forcing which forces $M_k(x', p)$? If yes, then do not perform that forcing; instead code $(x', m, k, p, 1)$;*

(iii) *if neither is possible, do nothing.*

A forcing is $\Gamma_{\alpha+1}$-allowable iff it is Γ_α-allowable and satisfies the requirement $\mathcal{R}(m, k, p, \alpha)$. At limit stages λ, set

$$\Gamma_\lambda = \bigcap_{\alpha<\lambda} \Gamma_\alpha.$$

Thus $\Gamma_{\alpha+1}$ is not defined before the iteration starts. It is created at the first stage where the construction meets a non-neutralized pair which cannot be made intersecting by the current class Γ_α. The rank α is then permanently attached to this pair, and all later decisions for this pair are made by asking forceability questions relative to the base class Γ_α.

Lemma 6.2 (Monotonicity and products)**.** *For $\alpha \leq \beta$ we have $\Gamma_\beta \subseteq \Gamma_\alpha$. Moreover, for each α, the class Γ_α is closed under finite products.*

Proof. Monotonicity is immediate from the definition: at successor stages we add an extra requirement, and at limit stages we take intersections.

Product closure is proved by induction on α. It holds for Γ_0 by the finite-use product lemma from the coding section: a finite product of finite-support iterations of coding blocks is again such an iteration, and each container is still used only finitely many times. Suppose product closure holds for Γ_α. Let $\mathbb{Q}_0, \mathbb{Q}_1 \in \Gamma_{\alpha+1}$, where $\Gamma_{\alpha+1}$ is obtained by adding $\mathcal{R}(m,k,p,\alpha)$. Since $\mathbb{Q}_0, \mathbb{Q}_1 \in \Gamma_\alpha$, their product belongs to Γ_α. The requirement $\mathcal{R}(m,k,p,\alpha)$ is formulated by universal instructions for future appearances of (m,k,p), and these instructions are preserved under taking finite products: in the product, a request is handled coordinatewise using the same Γ_α-forceability tests, and the resulting side codes are ordinary coding blocks. Hence $\mathbb{Q}_0 \times \mathbb{Q}_1 \in \Gamma_{\alpha+1}$. The limit case follows by intersecting the inductive hypotheses. □

6.3 The stage construction

We now define the finite-support iteration

$$\langle \mathbb{P}_\alpha, \dot{\mathbb{Q}}_\alpha \mid \alpha < \omega_2 \rangle.$$

At the same time we define the list of active requirements and hence the current class Γ_α. Suppose $\mathbb{P}_\alpha$, the active requirements, and Γ_α have already been defined. Let the bookkeeping present (x,p,m,k) in $W[G_\alpha]$.

(1) If (m,k,p) is already neutralized, let $\dot{\mathbb{Q}}_\alpha$ be trivial.

(2) If the current model already satisfies

$$\exists z \, \big(M_m(z,p) \wedge M_k(z,p)\big),$$

then declare (m,k,p) neutralized and let $\dot{\mathbb{Q}}_\alpha$ be trivial.

(3) If there is a Γ_α-allowable forcing $\mathbb{Q}$ which forces

$$\exists z \, \big(M_m(z,p) \wedge M_k(z,p)\big),$$

let $\dot{\mathbb{Q}}_\alpha$ be a name for the least such forcing in the fixed well-order of $H(\omega_2)$, and declare the pair neutralized after this stage.

(4) Suppose no Γ_α-allowable forcing can make the pair intersect. If the pair has not previously activated a requirement, add the requirement $\mathcal{R}(m,k,p,\alpha)$, thereby passing to $\Gamma_{\alpha+1}$. Now handle the present point x according to this new requirement: if some Γ_α-allowable forcing can force $M_m(x,p)$, append a coding block for $(x,m,k,p,0)$; otherwise, if some Γ_α-allowable forcing can force $M_k(x,p)$, append a coding block for $(x,m,k,p,1)$; otherwise do nothing.

(5) If (m, k, p) has already activated a requirement $\mathcal{R}(m, k, p, \rho)$ at some earlier stage $\rho < \alpha$, handle x by that requirement, i.e. ask the two side-forceability questions relative to Γ_ρ and code accordingly.

Whenever a coding block is appended, the container is chosen by the bookkeeping so that the finite-use condition remains true. In the simplest implementation the bookkeeping chooses a previously unused container. Thus every iterand is c.c.c., and $\mathbb{P}_{\omega_2}$ is a finite-support c.c.c. iteration.

Lemma 6.3 (Tails are allowable)**.** *Let $\alpha < \beta \leq \omega_2$. In $W[G_\alpha]$, the tail forcing $\mathbb{P}_\beta/G_\alpha$ is Γ_α-allowable.*

Proof. After stage α, every later stage is constructed so as to obey all requirements active at stage α. Later stages may add further requirements, but by monotonicity these only restrict the class further. Thus the tail is an iteration of coding blocks and neutralizing forcings satisfying every requirement defining Γ_α. The finite-use condition is preserved by the bookkeeping. Hence the tail belongs to Γ_α. □

Lemma 6.4 (Neutralization persists)**.** *If at some stage α the pair (m, k, p) is neutralized, then in every later extension by the iteration,*

$$\exists z \, \big(M_m(z, p) \wedge M_k(z, p)\big)$$

holds.

Proof. At the neutralizing stage, the intersection statement is made true either already in the current model or by an explicitly performed forcing. Since it is Σ_1 over the relevant $H(\omega_2)$-structure and the later forcing extensions preserve the relevant parameters and coding apparatus, it is upward absolute to all later stages. □

6.4 No bad placements after activation

We now prove the first key correctness lemma. It says that once a pair is first considered in the non-neutralized case, any later side placement for that pair is correct: an object coded on the m-side will never later lie on the k-side, and conversely. This is the place where finite product closure replaces the fixed-point machinery used in reduction and uniformization arguments.

Lemma 6.5 (No bad side placements)**.** *Suppose that (m, k, p) first activates a requirement $\mathcal{R}(m, k, p, \rho)$ at stage ρ. Thus no Γ_ρ-allowable forcing can force*

$$\exists z \, \big(M_m(z, p) \wedge M_k(z, p)\big).$$

Let $\beta \geq \rho$, and suppose that at stage β the construction codes $(x, m, k, p, 0)$, because some Γ_ρ-allowable forcing can force $M_m(x, p)$. Then no later stage of the actual iteration can make $M_k(x, p)$ true. Symmetrically, if the construction codes $(x, m, k, p, 1)$, then no later stage can make $M_m(x, p)$ true.

Proof. We prove the first assertion; the second is symmetric. The point to keep track of is that the side-forcing witnessing the possibility of putting x into the m-side is a forcing over the stage-β model, while the contradiction has to be a Γ_ρ-allowable forcing over the activation model $W[G_\rho]$.

Let $\mathbb{Q} \in \Gamma_\rho^{W[G_\beta]}$ witness the side-forceability at stage β, so that, in $W[G_\beta]$,

$$\mathbb{Q} \Vdash M_m(x, p).$$

Suppose toward a contradiction that the actual run of the iteration later makes $M_k(x, p)$ true. Then there is some stage $\gamma > \beta$ and a condition in $\mathbb{P}_\gamma/G_\beta$ which forces $M_k(x, p)$.

Now work over the activation model $W[G_\rho]$. First force with the actual intermediate tail

$$\mathbb{P}_\beta/G_\rho.$$

In the resulting stage-β model, force with the finite product

$$\mathbb{Q} \times (\mathbb{P}_\gamma/G_\beta).$$

Equivalently, over $W[G_\rho]$ consider the two-step forcing

$$(\mathbb{P}_\beta/G_\rho) * \big(\mathbb{Q} \times (\mathbb{P}_\gamma/G_\beta)\big).$$

By Lemma 6.3, the intermediate tail $\mathbb{P}_\beta/G_\rho$ is Γ_ρ-allowable. In the stage-β extension, both $\mathbb{Q}$ and $\mathbb{P}_\gamma/G_\beta$ are Γ_ρ-allowable, and by Lemma 6.2 their product is again Γ_ρ-allowable. Hence the displayed two-step forcing is a Γ_ρ-allowable continuation over $W[G_\rho]$.

In this continuation, the $\mathbb{Q}$-coordinate makes $M_m(x, p)$ true, and the actual-tail coordinate makes $M_k(x, p)$ true. Therefore this Γ_ρ-allowable forcing over $W[G_\rho]$ forces

$$\exists z \, \big(M_m(z, p) \wedge M_k(z, p)\big),$$

using $z = x$. This contradicts the defining property of the activation stage ρ, namely that no Γ_ρ-allowable forcing over $W[G_\rho]$ can make the pair intersect. □

Corollary 6.6 (No bad codes for an active pair)**.** *Let (m, k, p) activate at stage ρ. From stage ρ onward, every well-formed side code for this pair which is counted by the separator is an intentional code, and its side is correct. More precisely:*

(i) *if* $\mathrm{SepCoded}(x, m, k, p, 0)$ *holds because of a code created after stage ρ, then $M_k(x, p)$ never holds in the final model;*

(ii) *if* $\mathrm{SepCoded}(x, m, k, p, 1)$ *holds because of a code created after stage ρ, then $M_m(x, p)$ never holds in the final model.*

Proof. The no-unwanted-code lemma for finite-use containers, proved in the coding section, says that a well-formed code counted by the separator is one of the intentional codes produced by the iteration. Lemma 6.5 then gives correctness of the side. □

6.5 The separator associated with an active pair

Let (m, k, p) be a pair which is not neutralized in the final model. Let ρ be the stage at which it first activates a separation requirement. Define

$$D_{m,k,p} = \{x \subseteq \omega_1 \mid \mathrm{SepCoded}(x, m, k, p, 0) \text{ by a code created at or after } \rho\}.$$

Equivalently, the code contains the activation rank ρ in its header, so that codes created before the pair was first considered are ignored.

Lemma 6.7 (Coverage of the left side)**.** *If (m, k, p) is not neutralized and $M_m(x, p)$ holds in the final extension, then $x \in D_{m,k,p}$.*

Proof. Choose a stage $\beta \geq \rho$ at which the bookkeeping presents the tuple (x, m, k, p) after enough of the generic has been built so that some condition in the actual tail forces $M_m(x, p)$. Such stages occur unboundedly often by the bookkeeping. By Lemma 6.3, this tail is Γ_ρ-allowable. Therefore the m-side forceability question in the requirement $\mathcal{R}(m, k, p, \rho)$ has a positive answer at stage β, and the construction codes $(x, m, k, p, 0)$. Hence $x \in D_{m,k,p}$. □

Lemma 6.8 (Disjointness from the right side)**.** *If (m, k, p) is not neutralized, then*

$$D_{m,k,p} \cap \{x \mid M_k(x, p)\} = \emptyset$$

in the final extension.

Proof. If $x \in D_{m,k,p}$, then there is a counted side-zero code for (x, m, k, p) created after activation. By Corollary 6.6, $M_k(x, p)$ never holds in the final extension. □

Lemma 6.9 (Complexity of the separator)**.** *The set $D_{m,k,p}$ is Δ^1_1 in the final extension.*

Proof. The positive definition is the existence of a well-formed separation code with header $(m, k, p, 0, \rho)$, which is a Σ^1_1 condition by the coding section. The complementary definition says that no such well-formed intentional code exists. By the finite-use no-unwanted-code lemma, any apparent well-formed code with this header must be one of the intentional codes produced by the iteration. Thus failure of membership is expressed by the corresponding Π^1_1 no-code condition. Hence $D_{m,k,p}$ is Δ^1_1. □

Theorem 6.10 (Σ^1_1-separation)**.** *In the final extension, any two disjoint Σ^1_1 subsets of 2^{ω_1} are separated by a Δ^1_1 set.*

Proof. Let the two disjoint Σ^1_1 sets be defined by $M_m(x, p)$ and $M_k(x, p)$. If the pair (m, k, p) were ever neutralized, then by Lemma 6.4 the two sets would intersect in the final model, contrary to assumption. Hence the pair is not neutralized. Let ρ be its activation stage and form $D_{m,k,p}$ as above. By Lemma 6.7, the m-side is contained in $D_{m,k,p}$. By Lemma 6.8, $D_{m,k,p}$ is disjoint from the k-side. By Lemma 6.9, $D_{m,k,p}$ is Δ^1_1. Thus it separates the two sets. □

7 Remaining proof obligations

The simplified Cohen-trace presentation reduces the number of moving parts, but the following points still need to be written in their final form.

(1) Define precisely the preliminary model W and the coding of branch objects $b_{\eta,\xi,n,i}$ by subsets of ω_1.

(2) Specify the canonical set $Y(\eta, c, x, m, k, i)$ and prove injectivity of the code.

(3) Prove the finite-use no-unwanted-code lemma without suppressing the local model absoluteness details.

(4) Formalize the relativization of finite product closure from Γ_0 to Γ_α.

(5) Check that the final finite-support iteration preserves cardinals and has the intended continuum pattern.